VS-UVB

- **Komponente**

Eine Komponente ist eine modulare Einheit mit klar definierten erforderlichem und bereitgestellten Schnittstellen.

- **Architektur**

Die Architektur eines Systems beschreibt die Aufteilung in separate Komponenten und definiert

- die funktionalen Rollen der Komponenten
- ihre Beziehung untereinander
- wie Komponenten miteinander kommunizieren und
- wie sie auf ein Netzwerk von Computern abgebildet werden.

→ Geschichtete Architekturen

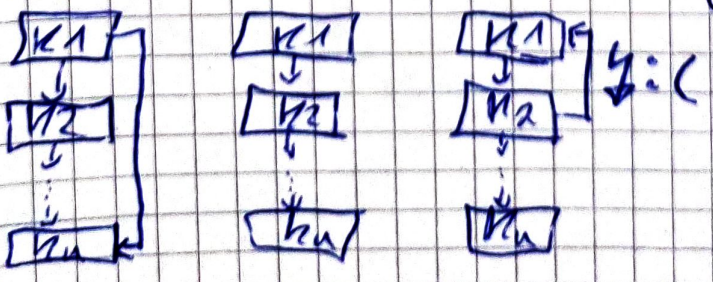
→ Einzelne Aspekte des Systems werden konzeptionell einer Schicht zugeordnet, d.h. Schicht = Komponente

→ Abhängigkeitsbeziehungen sind nur von einer Schicht zu einer tieferen Schicht erlaubt

→ Kommunikation mit Funktionsaufrufen oder NSGs

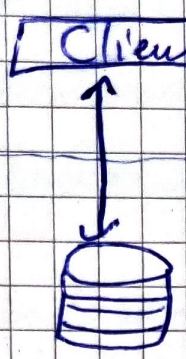
→ Häufiges Kopieren von Daten zwischen Schichten hat negative Auswirkung auf Skalierbarkeit

→ Leichter Austausch einer Implementierung einer Schicht



VS - MIB

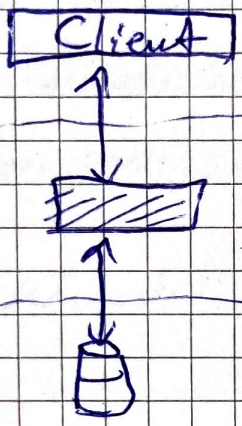
⇝ Zwei-Schichten-Architektur



- Benutzerschnittstelle
- Darstellung der Daten
- Anwendungslogik

- Anwendungslogik
- Konsistenz der Daten
- Persistente Verwaltung der Daten

⇝ Drei-Schichten-Architektur



- Benutzerschnittstelle
- Darstellung der Daten

- Anwendungslogik

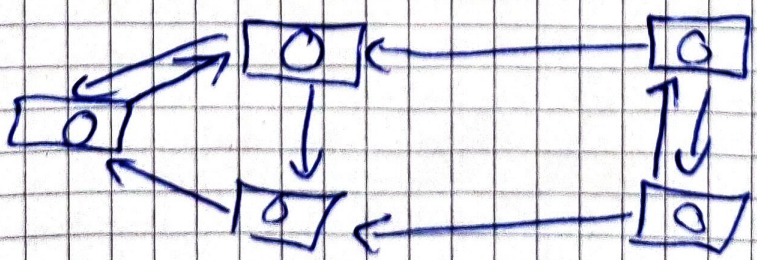
- Konsistenz der Daten
- Persistente Verwaltung der Daten

⇝ Objektbasierte Architekturen:

→ Lose Anordnung von Objekten (= Komponenten)

→ Kommunikation über Aufrufe von Methoden

→ Falls Objekte auf verschiedenen Rechnern, dann entfernte Aufrufe



VS-MVB

- **Pull Stil**

Beim Pull Stil erzeugt der Verbraucher den Informationsfluss.
Ein Verbraucher fordert explizit Daten vom Erzeuger an.
Verbraucher werden nicht informiert, wenn Erzeuger neue Daten hat.

↳ Client-Server Architekturen

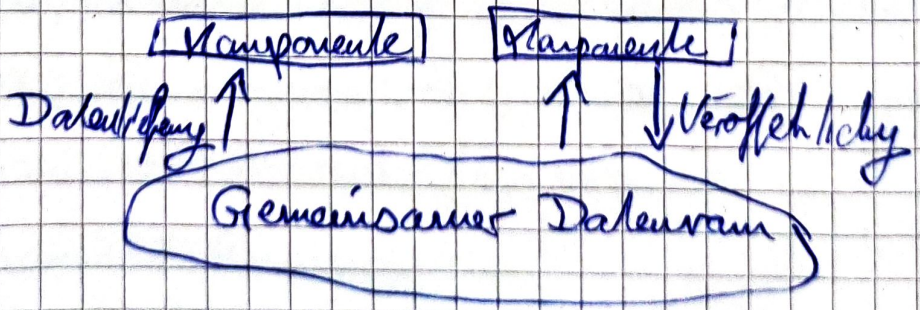
- **Push Stil**

Beim Push Stil steuert der Erzeuger den Informationsfluss.
Ein Erzeuger informiert explizit den Verbraucher, dass neue Daten vorliegen oder sendet Daten ungefragt direkt an Verbraucher.

↳ Verbraucher müssen sich bei Erzeuger registrieren

⇝ Datenzentrierte Architekturen:

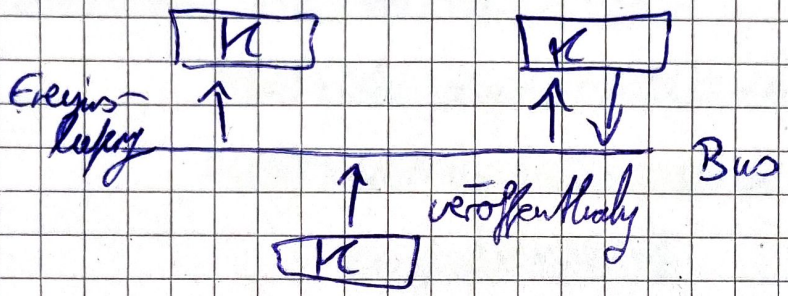
→ Komponenten kommunizieren mit gemeinsamen Datenraum
→ Keine Kopplung, Pull-Stil



VS-MVB

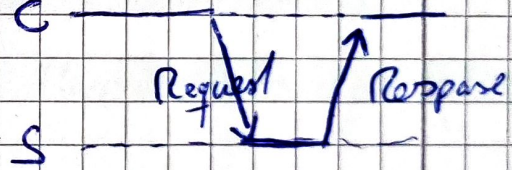
↳ Ereignisorientierte Architekturen:

- → Komponenten registrieren sich explizit bei anderen Komponenten und werden über deren Ereignisse informiert
- → kein polling, lose Kopplung, Push Stil
- → Publish - Subscribe Systems



→ zentrale Architekturen:

- → Client - Server Modell
- → Anfrage-/Antwortmodell
- → z.B. World Wide Web, E-Mail



↳ Dezentrale Architekturen

- → Komponenten sind alle gleich (Server & Client)
- → Alle Komponenten stellen gemeinsame Dienste bereit
- → Bessere Effizienz und Fehlertoleranz
- → Peer - to - Peer Systeme
  - ↳ Wie finden sich die Peers?

VS-MVB

→ Service-Oriented Architekturen (SOA):

→ Unterteilung nicht mehr aus Daten - sondern Funktionssicht

→ Aufteilung von Geschäftsprozessen in unabhängige Abläufe (Services)

→ Durch Zusammenspielen (Orchestrierung) von Services niedriger Abstraktionsebenen entstehen Services höherer Abstraktion

→ Vorteile: (+) Hohe Flexibilität

(+) Hoher Grad an Wiederverwendbarkeit

(+) Services können auch von externen Anbietern bezogen werden

→ Umsetzung als Web-Services via HTTP

→ lose Kopplung zwischen Services über Nachrichten / Ereignisse, statt enger Kopplung über entfernte Funktionsaufrufe

VS - KUB

⇝ Middleware

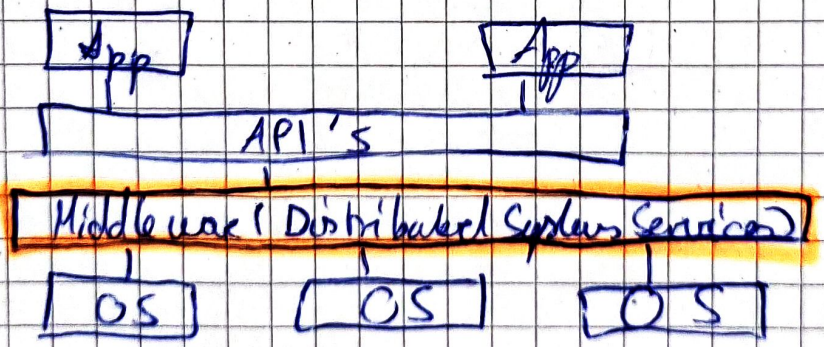
→ Bei der Realisierung VS treten Herausforderungen auf:

- → Heterogene Plattform
- → Fehlertoleranz
- → Skalierbarkeit

→ Herausforderungen sind unabhängig von einer speziellen Anwendung und sollen deshalb nicht für jede Anwendung auf Neue gelöst werden ⇝ LSG: Middleware

- **Middleware**

Middleware ist eine anwendungszentrale Softwareschicht zwischen Applikationen und Ebenen des Betriebsystems, welche die Unterschiede der Plattformen verdeckt.



→ Sorgt für ein einheitliches Programmiermodell, Programmierschnittstellen aller beteiligten Systeme werden auf eine Ebene gebracht ⇝ Heterogenität wird verborgen.

VS-NVS

→ Middleware:

→ Stellt Bausteine für den Aufbau von Komponenten bereit:

→ Entfernte Methodenaufrufe

→ Kommunikation zwischen Prozessen

→ Benachrichtigung über Ereignisse

→ Replikation gemeinsam genutzter Daten

→ Umsetzung durch Prozesse / Objekte in mehreren Computern

→ Middleware stellt keine Anwendungsfunktionalität bereit!!!

→ Middleware kann auch Dienste für Applikationen bereitstellen

→ Namensdienst

→ Transaktionsdienst

→ Persistenzdienst

→ Ereignisdienst

VS - KVB

- **Server**

Laufendes Programm, d.h. Prozess auf einem vernetzten Computer, der Dienst erbringt und ständig auf Anfragen wartet. Dienstbetreiber definiert Zugriffspunkt (IP-Adresse, Port), unter dem der Dienst erreichbar ist.

- **Client**

Anfragender Prozess auf einem vernetzten Computer, der einen Dienst nutzt.

→ Client-Server Architektur realisiert Pull Stil

→ Protokoll regelt Kommunikation: Request/Response Protokoll

⇝ Persistente Kommunikation

→ Eine versendete Nachricht wird durch das Kommunikationssystem so lange gespeichert bis die Nachricht zugestellt werden kann

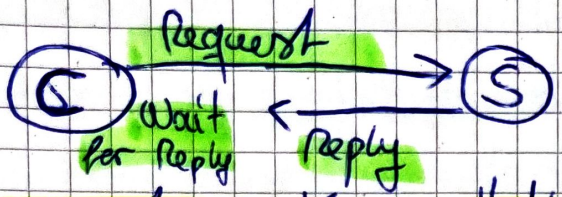
→ Speicherung beim Absender oder an unabhängiger Stelle

⇝ Transiente Kommunikation

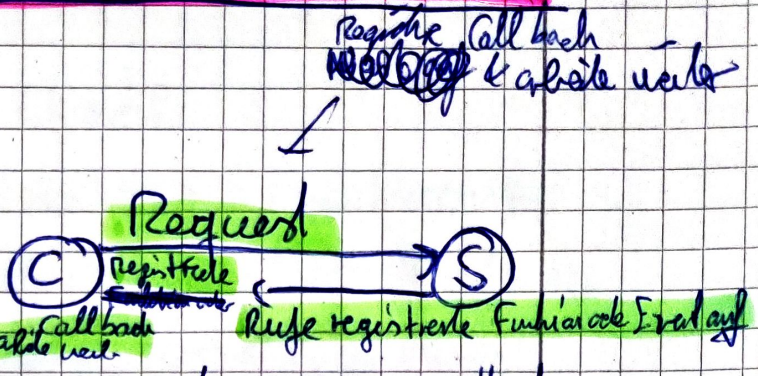
→ Eine Nachricht wird nur gespeichert, solange die sendende und empfangende Anwendung läuft.

VS - KV 15

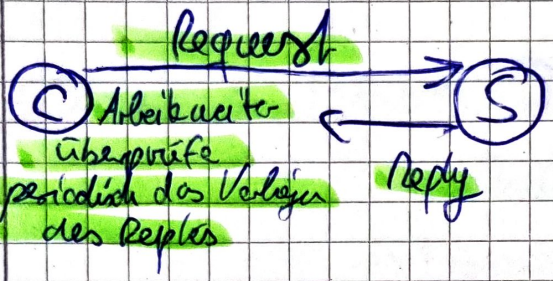
~> Transiente Interaktionsarten:



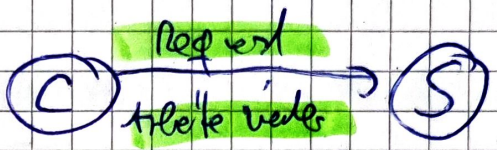
Synchrone Kommunikation



Asynchrone Kommunikation



Zurückgestellte synchrone Kommunikation



Ein-Weg-Kommunikation

~> Request / Response Protokoll:

-> Synchrone Kommunikation:

1) Server wartet ständig auf Anfragen (IP-Adresse, Port)
2) Client stellt Verbindung zu Server her
3) Client schickt Anfrage und wartet auf Antwort
4) Server empfängt Anfrage, bearbeitet sie und schickt Antwort zurück
5) Client empfängt Antwort und gibt sie an Anwendung weiter
6) Client schließt Verbindung

-> Wie identifiziert Client den Server? -> Bekannte Adresse -> logische Namen (Vermittler)

-> Wie ist Anfrage beschrieben? -> Feste Anfragen -> Anfrage enthält Parameter

VS - KVP

→ Spezifikation der Anfrage durch Client:
→ Auf der Ebene von TCP wird eine Folge von Bytes übertragen
→ Anfrage- und Antwortdaten müssen so kodiert werden, dass Empfänger eine eindeutige Dekodierung vornehmen kann.

→ Leistungskriterien für C/S-Architekturen:

→ Antwortzeit: → Zeitspanne zwischen Absenden der Anfrage und vollständigem Empfangen der Antwort
→ Datenübertragung in Übertragungsmedium, Wartezeit, Bearbeitungszeit

→ Durchsatz: → Maß für Menge an Datenpro Zeiteinheit die über Netzwerk übertragen und bearbeitet werden können
→ Anzahl pro Zeiteinheit

→ Latenzzeit: → Dauer der Übertragung von Daten von Client zu Server, Antwortzeit ≥ Latenzzeit
→ unabhängig von Größe der Anfrage und Bearbeitungszeit des Servers

→ Jitter: → Abweichung der Latenzzeit oder Antwortzeit vom Mittelwert

→ Last: → Belastung des Systems

→ Lastabhängigkeit: → Abhängigkeit der Antwortzeit von der Last

→ Dienstgüte (Quality of Service, QoS):
→ Fähigkeit eines Systems, die Einhaltung von Vorgaben bzgl. Leistungskriterien zu garantieren.
→ Umsetzung: → Priorisierung des Datenverkehrs
→ Garantierte Bereitstellung von Rechen- und Kommunikationsressourcen
→ TCP/IP ist best effort.

→ Service-Level-Agreement (SLA):
→ Vereinbarung zw. Auftraggeber und Dienstleister bzgl. QoS

VS-HÜB

→ Iterativer Server

- → Anfragen werden sequentiell bearbeitet: Single threaded Server
- → Treffen Anfragen während der Anfragebearbeitung ein, so werden diese abgewiesen / gepuffert oder ignoriert.
- → Nur bei einfachen Diensten mit kurzer Bearbeitungszeit sinnvoll

socket()
↓
bind()
↓
listen()
↓
[→ accept()
↓
Read/Write
↓
close()]

→ Nebenläufiger Server

- → Mehrere Anfragen werden gleichzeitig bearbeitet
- → Sinnvoll bei langen Aufträgen
- → Realisiert → Ein Thread/Prozess pro Anfrage
- → Vorteile: (+) Mittlere Antwortzeit pro Anfrage wird reduziert
  (+) Bessere HW Auslastung

socket()
↓
bind()
↓
listen()
↓
[accept()
↓
fork() → (Child Process) Read/Write → close()]
↓

VS - VL VB

→ Nebenläufiger TCP Echo-Server

```
public class Dispatcher :
    ServerSocket sock = new ServerPort(PORT);
    while (true) {
        Socket cSocket = sock.accept();
        ArbeiterThread at = new ArbeiterThread(cSocket);
        at.start();
    }
```

→ Dienst Bereitstellung durch mehrere Server

- → Dienste werden mehrfach in separaten Computern realisiert
- → Vorteil: (+) Erhöhung der Verfügbarkeit, Lastverteilung, Ausfallsicherheit
- → Unterschied: Daten verteilen (aufteilen), Daten replizieren (kopieren)
- → Kombination: → Statische Objekte verteilen
  → Dynamische Objekte replizieren

↳ Nicht transparent:

- → Nachteil: Clients müssen die ~~Anfragen~~ Adressen aller Server kennen
- → Höher Aufwand bei Hinzufügen bzw. Entfernen von Servern

↳ Transparent:

- → Vorteile: (+) Clients müssen nur eine einzige Adresse kennen
  (+) Hinzufügen bzw. Entfernen von Servern ist einfach
- → Realisiert durch Verteilserver

VS-VNS

Verteilserver

→ Verteilserver leitet Anfragen nur weiter

→ Client muss Aufteilung der Daten nicht kennen

→ Verteilserver kann Cache umsetzen

→ Lastverteilung durch Verteilserver

- → Zufallsprinzip
- → Rundlauf-Verfahren
- → Lastabhängig
- → Nächstliegender Server

→ Probleme: → Single Point of failure

→ Überlastung des Verteilservers

- **Replikation**

Replikation bezeichnet die mehrfache Speicherung derselben Daten an unterschiedlichen Orten und deren Synchronisation.

→ Synchrone Replikation

Änderungen sind nur dann erfolgreich abgeschlossen, wenn sie auf allen Replikaten durchgeführt wurden, d.h. Daten sind jederzeit konsistent.

→ Asynchrone Replikation

Zwischen der Bereitstellung der primären und der replizierten Daten kann Zeit vergehen, d.h. Daten nicht jederzeit konsistent

Verteilte Daten	Replizierte Daten
Transparent	Nicht transparent

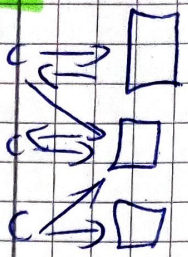
VS-KVB

⇝ Verteilte Daten: Nichttransparente Aufteilung

→ Mehrere Server bieten Dienst an, jeder Server verwaltet eigene Daten

→ Aufteilung der Server nicht transparent

↳ Clients muss wissen, welcher Server welche Daten hat und entsprechende Adresse kennen

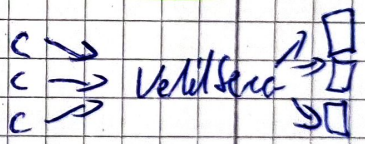


⇝ Verteilte Daten: Transparente Aufteilung

→ Zugriff über Verteilserver

↳ Nach außen erscheinen Server wie ein einzelner Server

↳ Aufteilung ist transparent, d.h. Adressen der Server mit den Daten bleiben verborgen



⇝ Replizierte Daten

→ Mehrere Server bieten Dienst an, jeder verwaltet die gleichen Daten (Replikation)

→ Realisierung mit/ohne Verteilserver

→ Unterschied zu verteilten Daten:

→ Wahrung der Konsistenz bei schreibenden Zugriffen

→ eg: WWW

↳ Anwendung wird auf viele Web-Server repliziert oder Daten werden aufgeteilt

↳ Front-End Server nimmt Verteilung vor

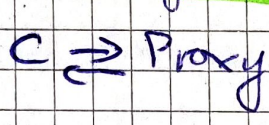
VS - KVB

- **Proxy**

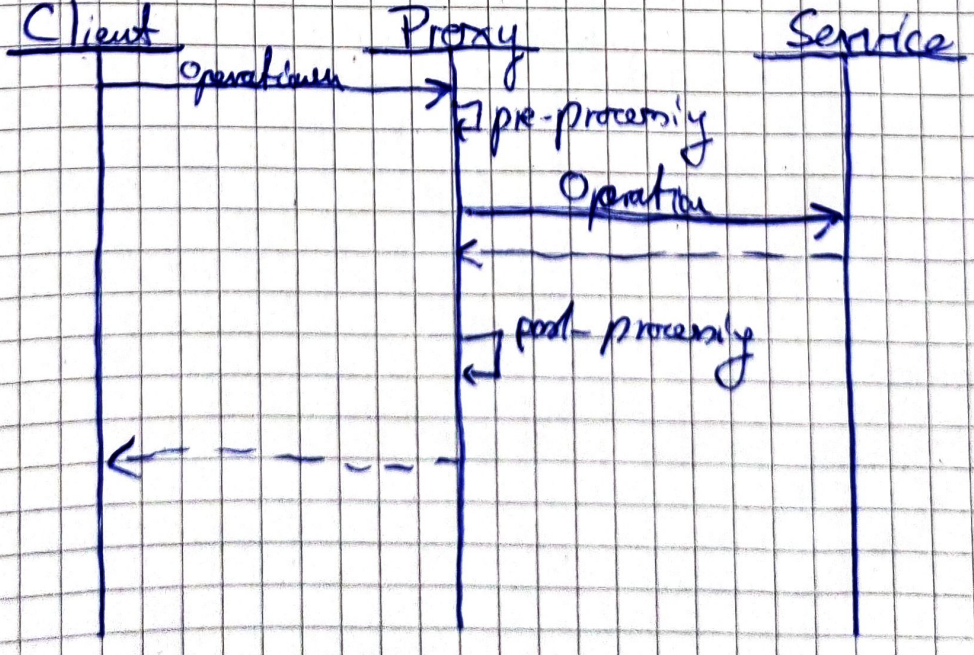
Ein Proxy hat den Zweck, den Zugriff auf ein Objekt mit Hilfe eines vorgelagerten Stellvertreterobjekt zu kontrollieren.

↳ Erweiterung des Konzeptes des Verteilservers
- → Proxy kann mehr als nur Anfragen weiterleiten
- → Proxy bietet Clients einer Komponente einen Repräsentanten für die Komponente

↳ Vorteile: (+) Entkopplung von Clients von Ort des Servers
(+) Proxy befreit Client/Server von nicht unmittelbar zur Durchführung der Aufgabe benötigten Tätigkeiten



↳ Server stellt Operationen zur Verfügung
↳ Proxy implementiert gleiche Schnittstelle
↳ Proxy leitet Zugriffe weiter zu Server



VS - KVB

Proxy - Arten:

→ Remote Proxy

Clients wird Netzwerkadressierung und Interprozesskommunikation abgenommen. → Client nutzt API

→ Protection Proxy

Komponenten werden vor unautorisiertem Zugriff geschützt,
Proxy überprüft Zugriffsrechte

→ Cache Proxy

Clients teilen sich Ergebnisse von entfernten Komponenten
↳ Proxy hat Cache-Funktionalität
↳ Caches sind Zwischenspeicher, die näher bei Clients als die eigentlichen Objekte liegen

→ Smart Proxy

Proxy übernimmt Teile der Serverimplementierung

VS - KVB

	Fat - Client	Thin - Client
Client:	Präsentation Applikation Daten	Präsentation
Server:	Daten	Applikation Daten

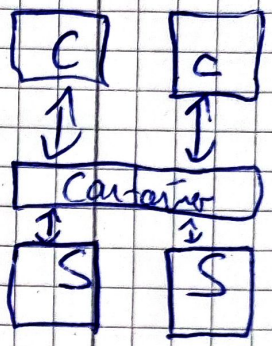
→ Implementierung von Anwendungsservern

→ Anwendungsservers werden oft als Container implementiert

→ Container → verwalten Komponenten
→ verbergen Komplexität

→ Komponenten implementieren vorgegebene Schnittstelle
↳ Container muss Komponenten vorher nicht kennen

→ Container → bieten Laufzeitumgebung an
→ Stellen Dienste zur Verfügung

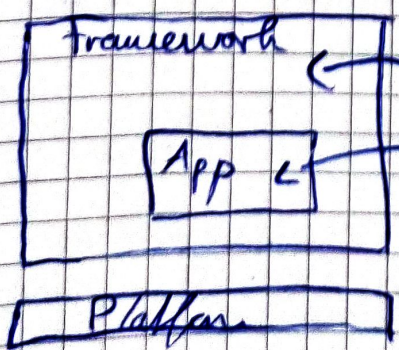


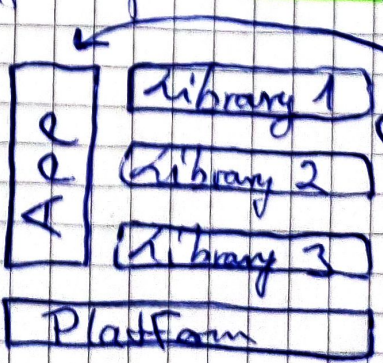
- **Bibliothek**

Eine Bibliothek ist eine Ansammlung von Funktionen mit Lösungen für thematisch zusammengehörende Probleme.

- **Framework**

Ein Framework stellt für Anwendungen eine wiederverwendbare, gemeinsame Struktur zur Verfügung. Es definiert Schnittstellen, die von Anwendungen implementiert werden.





VS-KUB

Hollywood Prinzip

- Unterschied:
  - → Anwendung ruft Funktion aus Bibliothek auf
  - → Framework ruft Anwendung auf
- → Framework:
  - → Anwendung steuert nicht den Kontrollfluss
  - → Steuerung des Kontrollflusses wird an Framework abgegeben
    - ↳ Inversion of Control
- → Vorteile: Frameworks implementieren ein abstraktes Design

Dienste von Frameworks:

- → Lebenszyklus Verwaltung
  - Objekte initialisieren, konfigurieren, ausführen & zerstören
- → Namensdienst
  - Objekte registrieren, nachschlagen und deregistrieren
- → Konfigurationsmanagement
  - Objekte mittels Konfigurationsdateien durch IoC implementieren
- → Sitzungsverwaltung

VS-KVB

- HTTP: Hypertext Transfer Protocol
  - Generisches Client-Server Protokoll
    - Austausch von Daten in beliebigen Formaten
  - Adressierung in TCP:
    - TCP adressiert Anwendung auf Rechner
    - IP-Adresse des Servers und port der Anwendung
  - Adressierung in HTTP:
    - HTTP addressiert Dokument auf Rechner
    - Adressierung mit URLs
  - URL-Format:
    - Servername mit Port (www.sale.de)
    - Pfadangabe (zB /dienst/shop)
    - Parameter (var1=alles & id=21)

- Nachrichtenformat
  - HTTP Nachrichten sind in Head und Body aufgeteilt
  - ~~Head in Hebräer~~ HTTP Head ist in ASCII codiert, Rest beliebig kann
  - Head und Body: Gleiche Struktur für Anfrage & Antwort
  - Ende des Head: HTTP: Erste Leerzeile
  - Struktur des Head
    - Zeilenorientiert: Pro Zeile ein Name-Wert Paar
    - Einzelne Zeile wird Header genannt
  - Struktur des Body:
    - Keine Vorgabe, beliebiges Format
    - Header enthält Angaben zur Länge des Body

VS-KVB

⇝ Semantik der Anfrage

→ Semantik wird durch erste Zeile im Head beschrieben

→ HTTP Kommandos:

- GET: Abfragen von Informationen ohne Serverzustand zu ändern
  - ↳ Lesen einer Ressource
- PUT: Ablegen von Ressourcen, ändert Serverzustand
  - ↳ Speichern von ~~Ressourcen~~ Daten
- POST: ~~Ablegen von~~ + Hinzufügen von Informationen, ändert Serverzustand
  - ↳ Daten an File anhängen
- DELETE: Löschen von Ressourcen, ändert Serverzustand

⇝ Ablauf einer Anfrage

1) Client baut TCP-Verbindung zum Server auf
2) Client sendet HTTP-Anfrage an Server
3) Server schickt HTTP-Antwort an Client
4) Client & Server schließen TCP-Verbindung

⇝ Beispiel:

Anfrage: GET client/index.html * HTTP/1.1

Antwort: HTTP/1.1 ** 200 OK

<html> ...

* Cookie: name=value,...

** Content-Length: 1900
Set-Cookie: name=value,...

VS-HÜB

→ HTTP Antwort

→ Server informiert Client über Status der Anfrage

HTTP/1.0 Statuscode Statustext

→ Statuscodes

→ 1xx : Information

→ 2xx : Anfrage konnte erfolgreich verarbeitet werden

→ 3xx : Ressource unter anderer URL erreichbar

→ 4xx : Fehler auf Client-Seite verursacht

→ 5xx : Fehler auf Server-Seite verursacht.

→ Angaben zur Kodierung des Body im Header Content-Type

→ MME-Types : image/jpeg, text/html

→ Client nutzt Content-Type zur Anzeige der Daten

→ Parameter

→ HTML stellt Formulare zur Eingabe von Parametern bereit

→ Namen und Inhalte der Felder sind Bestandteil der Anfrage

→ Browser kodiert die Angaben in eine einzige Zeichenkette

```
<form action="..." method="POST">
Name: <input name="username" />
EMail: <input name="email" />
Text: <textarea name="text"/>
<input type="submit" value="send form data" />
```

→ Daten im Body der Anfrage

username=Stefan+M%0Fcller&email=stefan&text=hüs

MIME-Type : application/x-www-form-urlencoded

VS-KVB

→ Web-Anwendungen

→ Im WWW werden i.d.R. Hypertexte im HTML-Format angefragt

→ Dokumente werden oft erst beim Eintreffen der Anfrage durch eine Anwendung generiert.

→ Anwendungen sind Programme, die HTML-Dokumente generieren

→ Programme werden über URLs identifiziert

→ Server rufen Programme bei Anfrage an URL mit den übergebenen Parametern auf und geben generiertes Dokument zurück

→ Parameter werden als kodierter String übergeben

→ Programm nimmt Dekodierung in geeigneten Datentypen vor

→ Kodierung der Parameter

→ Zur Vereinfachung der Übertragung strukturierter Daten wurden Datenaustauschformate entwickelt

→ eg: JSON → einfaches textbasiertes Datenaustauschformat

MIME-Type: application/json

```
{
  "coord" : {"lon" : -0.13, "lat" : 51.51},
  "sys" : {"message" : 0.5, "country" : "GB", "sunrise" : 123},
  "dt" : 123,
  "cod" : 200
}
```

JSON

VS - HÜB

Skalierbarkeit von HTTP

- Nachteile HTTP/1.x
  - Header werden unkomprimiert übertragen
  - Header aufeinanderfolgender Nachrichten unterscheiden sich kaum
  - Für jede Anfrage wird eine neue TCP Verbindung geöffnet
  - Anzahl offener Verbindungen durch BS begrenzt

- Optimierung von HTTP/1.1
  1) Persistente Verbindungen → keep-alive header
     - Verbindungen werden erst nach Timeout geschlossen
     - Problem: Erste Antwort kann weitere Antworten verzögern
       head-of-line blocking
  2) Pipelining
     - Mehrere HTTP-Anfragen werden einem einzigen Socket übergeben, ohne auf die Antwort zu warten
     - Server kann Anfragen parallel bearbeiten
     - Antworten werden in der gleichen Reihenfolge wie Anfragen gesendet.

VS - KV

→ HTTP/2

↳ Ziel: Optimierung der Datenübertragung (Bandbreite, Latenz)

-> Zusammenfassung mehrerer Anfragen (multiplexing)

-> Datenkompressionsmöglichkeiten (Header-Komprimierung)

-> binär kodierte Übertragung

-> out-of-order delivery

-> Server-initiierte Datenübertragungen (push-Verfahren)

↳ Tendenz: Alle Daten verschlüsselt übertragen

↳ Neue Schicht zwischen TCP und HTTP API

↳ Semantik von HTTP bleibt erhalten, nur Kodierung ändert sich

↳ Transparente Aufteilung von HTTP Nachrichten in Frames und Übertragung in Strömen

↳ Alle Kommunikation zwischen einem Client und einem Server läuft über eine TCP-Verbindung

↳ HTTP/2 bildet HTTP auf Austausch binär kodierter Frames ab

VS-KÜB

- **Sitzung (Session)**
  Eine Sitzung besteht aus einer Folge von Interaktionen zwischen einem Client und einem Server.

↳ Der Zustand einer Sitzung wird durch die Zustandsdaten beschrieben

- **Zustandslosigkeit**
  Ein Protokoll ist zustandslos, falls mehrere Anfragen (auch desselben Clients) grundsätzlich voneinander unabhängig sind, d.h. Anfragen werden ohne Bezug zu früheren Anfragen behandelt und es werden keine Zustandsdaten verwaltet.

↳ HTTP ist ein zustandsloses Protokoll

→ Wo werden Zustandsdaten gespeichert?

→ A) Speicherung beim Client:
- Client schickt mit jeder Anfrage alle Zustandsdaten an Server
- Server Anwendung aktualisiert Zustandsdaten und schickt Daten mit Antwort zurück
- Server ist vollkommen zustandslos
- Nachteile: (-) Client kann Daten ändern, erhöhter Datenverkehr
- Vorteile: (+) Server kann zwischen Anfragen neu gestartet werden

VS - KW3

→ Wie werden Daten übertragen:

→ URL Parameter, URL rewriting
↳ Server-Anwendung hängt Daten an jede URL in Antwortseite
↳ Nachteil: Größe der Daten begrenzt
↳ eg: http://sf.de/req.jsp?isb=978-11-41&anzahl=1

→ Hidden fields
↳ Server-Anwendg schreibt Daten in Antwortdokument
↳ hidden fields in HTML Form-Elemente

→ Cookies
↳ HTTP Header

→ B) Speicherung auf Server
→ Server muss eingehende Anfrage gespeicherten Daten zuordnen
→ Sitzungskennung (Session-ID) identifiziert Sitzung
↳ Zustandsbehaftete Protokolle: gestaltet als Teil des Protokolls
↳ Zustandslosen Protokoll: Anwendungen sind dafür verantwortlich
→ Server weist jede Anfrage anhand v. Session-ID einer existierenden Sitzung zu
→ Enthält Anfrage keine Session-ID, dann beginnt neue Sitzung
→ Server legt neue Sitzung an, erzeugt neue Session-ID und schickt diese in Antwort zurück
→ Client schickt Session-ID mit jeder weiteren Anfrage an Server

VS-KVB

→ Session-IDs

→ Übertragung der Session-ID bei HTTP häufig mit Cookies

→ Session-IDs müssen schwer zu erraten sein

→ Session-IDs werden heute zur Personalisierung verwendet

↳ Dauerhafte Personenerkennung werden als Session-IDs verwendet

↳ Permanente Speicherung von Session-IDs durch Web-Browser

↳ Durch Weitergabe von Kennungen an andere Unternehmen werden Nutzerprofile erstellt

↳ Privatsphäre gefährdet.

→ Zustandsdaten in Datenbank

→ Sitzungsnummer ist Schlüssel für Datenbanktabelle

↳ Löschen nach Timeout

VS-KUB-Ex 3

(3-1)

1) Pull & Push Kommunikation?

Push Stil:

Beim Push Still steuert der Erzeuger den Informationsfluss.
Ein Erzeuger informiert explizit den Verbraucher, dass neue Daten vorliegen oder sendet Daten ungefragt direkt an Verbraucher

Pull Stil:

Beim Pull Stil steuert der Verbraucher den Informationsfluss.
Ein Verbraucher fordert explizit Daten von Erzeuger an.

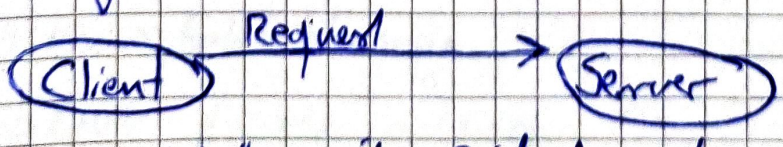
2) Cloud Computing?

Cloud Computing:

Beim Cloud Computing werden IT-Ressourcen wie Speicherplatz, Rechenleistung oder Anwendungssoftware als Dienstleistung über das Internet bereitgestellt. Ziel ist eine unbegrenzte horizontale Sicherung mit hoher Elastizität

3) Ein-Weg-Kommunikation?

Ein-Weg-Kommunikation:



arbeite weiter, Rückgabewert wird nicht benötigt

VS - KW/D - Gx 3

(3.1)

4) Framework vs Bibliothek?

Ein Entwickler ruft innerhalb seines Codes die Funktionen der Bibliothek auf.

Hingegen ruft das Framework vorgesehene Funktionen selbständig auf

5) Warum wird HTTP textbasiertes Protokoll genannt?

Da Header ASCII ~~code~~ kodiert ist.

6) Was ist ein Cookie und welche Verwndg haben Cookies?

Cookie ~> HTTP Header

-> speichert zum Beispiel Session ID

oder allgemein wird zum Datenübertragen verwendet

-> Speichert Benutzerzustand

7) Vier Technologien zur Steigerung der Skalierbarkeit von HTTP.

-> Persistente Verbindung

-> Piplinening

-> Daten kompressionsmöglichkeiten

-> Zusammenfassung mehrerer Anfragen (multiplexing)

-> binär kodierte Übertragung

-> Out-of-order delivery

VS-NVB-Ex 3

(S-1)

8) Separation of Concerns?

Separation of Concerns

is a design principle for separating a pc program into distinct sections such that each section addresses a separate concern.

VS-KVB-Ex 3

(3.2)

Jede Sekunde eine Messung.

1) Eine Anzeigetafel soll mir den aktuellen Wert anzeigen
2) Eine externe Datenbank will jede Sekunde den aktuellen Wert protokollieren
3) Der aktuelle Wert soll bei Bedarf in einer App angezeigt werden
4) Eine externe Einheit soll bei Überschreitung eines Grenzwertes einen Alarm auslösen

a) Netzwerklast, wenn viele MSG pro Zeiteinheit.

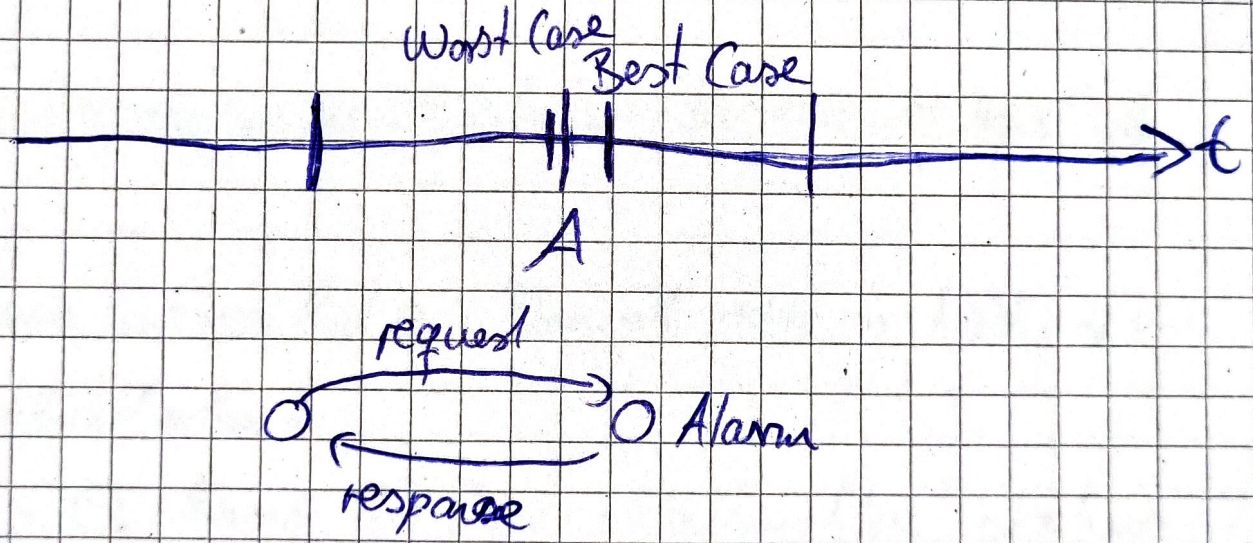
	PULL	PUSH	Vorteil
1)	7200 MSG/Std	3600 MSG/STD	PUSH
2)	2 MSG/Std	1 MSG/STD	PUSH
3)	2·n MSG/Std	3600 MSG/STD	PULL
4)	7200 MSG/Std	3600 MSG/STD	PUSH

VS - HÜB - 6x3

3.2

b) Kateuer, die Zeit vom Feststellen der Überschreitung der Temperatur bis zur Anzeige der Grenzüberschreitung.

Wie hoch muss das Abfrageintervall der Alarmzentrale bei Pull-Kommunikation sein, damit der Alarm im Mittel 1 Sekunde später ausgelöst wird?



$\frac{\text{Best} + \text{Worst}}{2} = 1\,s \Rightarrow$ Worst Case $2\,s \triangleq$ Aufzeichnungsintervall

VS - KVB - Ex 3

(3.3) Implementierung von Push-Kommunikation

Während die Adressen des Servers den Clients im Allgemeinen bekannt ist und so eine Pull-Kommunikation einfach durchgeführt werden kann, gilt dies nicht umgekehrt.

→ Welche prinzipiellen Möglichkeiten gibt es dennoch eine Push-Kommunikation durchzuführen?

→ Clients müssen sich beim Server registrieren

→ Welchen Einfluss hat eine Firewall oder ein NAT auf die Durchführbarkeit?

→ Es kann die Push-Nachrichten blockieren :(

→ Menge der Clients sehr große Probleme?

→ Viele Nachrichten

→ Schwer alle Clients zu speichern

VS-KVB-Gr 3

(3-0) HTTP

1) Siehe Exercise 3

Welche Zeichenkette wird bei der Parsierung der Parameter erzeugt?

name = Bartholomaus_Müller & e-mail = bati@mueller
& geschlecht = m & newsletter = yes

2) In welchen Teil der HTTP-Nachricht wird die Daten am Server gesendet?

→ HTTP-Body

```
POST /register.php HTTP/1.1
Host: www.server.org
"HTTP-Headers"
Content-Type: application/x-www-form-urlencoded
Content-Length: 83

name = Mynare & email = mynail
```

VS - HÜB - Ex3

(3-5) JSON

```
JSONObject book = new JSONObject();
    book.put("Titel", "Bla");
    book.put("Auflage", 3000);
    book.put("Preis", 39.80);
    book.put("Inhalt", null);

JSONObject autor = new JSONObject();
    autor.put("Vor", "Volker");
    autor.put("Nach", "Turau");
    book.put("Autor", autor);
```

→ book.toString():

```
{ "Title" : "Bla",
  "Auflage" : 3000,
  "Preis" : 39.90,
  "Inhalt" : null,
  "Autor" : { "Vor" : "Volker", "Nach" : "Turau" }
}
```

← "bestseller" : true,

VS - KVB - Ex 3

(3-6) Sessions

Session IDs dienen dazu zu erkennen, dass verschiedene Anfragen vom selben Benutzer stammen. Normalerweise wird zu diesem Zweck bei der ersten Anfrage eines Benutzers eine Zufallszahl aus einem sehr großen Wertebereich erzeugt, die ihm vom Server zugewiesen wird, und die der Client bei jeder Anfrage mitsendet.

Ein Entwickler will die Erzeugung einer Zufallszahl für jede neue Sitzung vermeiden. Um Zeit zu sparen, legt er bei Systemstart einen Pool von Zufallszahlen an. ~~[illegible]~~

Nachteile:
- → Begrenzte Session IDs
- → Brute Force kann einfach alle SIDs finden
- → Privatsphäre des Nutzers nicht sicher
- → SIDs leicht zu erraten
- → Sogar Daten von "alten Benutzern" kann gesehen werden
- → DOS Attacks.

VS - Kurs - Ex 3

(3.7) Replikationen

Nicht repliziertes Dokument mit der Größe von s Byte r mal pro Sekunde von unabhängigen Rechnern angefordert wird.

Das Dokument wird auf k unterschiedlichen Servern repliziert und w mal pro Sekunde aktualisiert. Es wird vorausgesetzt, dass Aktualisierungen zu jedem Replikat einzeln weitergeleitet werden. Weiterhin befinde sich in der Nähe jedes Abfragers ein Server mit einem Replikat, so dass in diesem Fall keine Kosten zur Abfrage mehr anfallen.

Ab wann sind die Kosten (Bandbreite [B/s]) für das Verfahren mit der Replikation kleiner als die Kosten für das nicht replizierte Verfahren?

$s \cdot r$ > $s \cdot k \cdot w$

r > $k \cdot w$

VS-KVB-Praxis 3

① URL - Java Klasse für HTTP Anfragen

```
URL
getResponseCode() {
    URL url = new URL("http://example.com/index.html");
    URLConnection urlCon = url.openConnection();
    int status_code = ((HttpURLConnection) urlCon).getResponseCode();
}
```

② JSON with GSON

```
import com.google.gson.Gson;
Gson gson = new Gson();
String jsonString = gson.toJson(myObject);
MyClass mc = (MyClass) gson.fromJson(jsonStr, MyClass.class);
```

③ HTTP

GET Method: GET is used to request data from a specific resource.

eg: GET /index.php?sucht=test HTTP/1.1
[...]

POST Method: POST is used to send data to a server to create/update a resource

eg: POST /test/demotrans.php HTTP/1.1
Host: www.google.de
[...]
(Body) name1=Mike&name2=Ursula